

## SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

### ASSESSMENT TOOL 3 – Comparative Analysis

|                  |                                                                              |               |                                          |
|------------------|------------------------------------------------------------------------------|---------------|------------------------------------------|
| Course Code:     | CSA0609                                                                      | Course Title: | Design and Analysis of Algorithms        |
| CO Assessed:     | CO4: Articulation (BL4, BL5)                                                 | Assessment:   | Assessment Tool 3 – Comparative Analysis |
| Weightage:       | 10% of CIA                                                                   | Total Marks:  | 100                                      |
| CO4 Statement:   | Solve optimization problems using dynamic programming and greedy strategies. |               |                                          |
| Student Name:    | Ranjitha R                                                                   | Reg. No.:     | 192421285                                |
| Section / Batch: | Information Technology                                                       | Date:         | 25/08/2026                               |

#### Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

| Question Type        | Focus Area                                                                                   | Questions |
|----------------------|----------------------------------------------------------------------------------------------|-----------|
| Comparative Analysis | Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems. | Q1        |

#### SECTION A – Comparative Analysis

##### Q31. Job Sequencing vs Huffman Coding (Greedy Comparison)

Given jobs with deadlines and symbol frequencies, solve Job Sequencing and Huffman Coding. Compare greedy strategies used. Analyze correctness and optimality. Evaluate time complexity. Discuss real-world applications. Generate insights on greedy paradigm. Instructions: Solve the assigned problem, show the algorithm/steps clearly, analyze correctness, time and space complexity, and provide the requested comparison or insights.

#### Code of Conduct

*I Ranjitha R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



## RUBRICS FOR CALCULATION

| Comparative Analysis            |           |                                                                                               |                                                                  |                                         |                                               |
|---------------------------------|-----------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------|-----------------------------------------------|
| Criteria                        | Weightage | Level 4 (Exemplary)                                                                           | Level 3 (Proficient)                                             | Level 2 (Developing)                    | Level 1 (Beginning)                           |
| Selection of Studies/Parameters | 20%       | Selects highly relevant and diverse studies with well-defined comparison parameters           | Selects relevant studies with minor gaps in parameter definition | Limited or partially relevant selection | Poor or inappropriate selection of studies    |
| Comparison & Differentiation    | 25%       | Clearly compares multiple aspects (methods, results, limitations) with strong differentiation | Good comparison with minor gaps                                  | Basic comparison with limited depth     | No clear comparison; mostly descriptive       |
| Critical Evaluation             | 25%       | Critically evaluates strengths, weaknesses, and implications with strong justification        | Provides evaluation with some justification                      | Limited evaluation; lacks depth         | No critical evaluation provided               |
| Synthesis & Insight Generation  | 20%       | Generates meaningful insights and conclusions from comparisons                                | Some insights derived from comparison                            | Limited or unclear insights             | No insights generated                         |
| Clarity                         | 10%       | Information is clearly presented (tables/figures) with logical structure                      | Generally clear with minor issues                                | Presentation lacks clarity or structure | Poor presentation and difficult to understand |

### Marks Evaluation:

| Criteria                        | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---------------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Selection of Studies/Parameters | 20% |                     |                      |                      |                     |
| Comparison & Differentiation    | 25% |                     |                      |                      |                     |
| Critical Evaluation             | 25% |                     |                      |                      |                     |
| Synthesis & Insight Generation  | 20% |                     |                      |                      |                     |
| Clarity                         | 10% |                     |                      |                      |                     |
| <b>Total Marks (100):</b>       |     |                     |                      |                      |                     |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name:             | Name:              | Name:              |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

## JOB SEQUENCING VS HUFFMAN CODING — GREEDY COMPARISON

Both Job Sequencing with Deadlines and Huffman Coding are classic Greedy Algorithm problems. However, they use the greedy strategy in different ways.

### 1. Problem Overview

| Feature             | Job Sequencing                  | Huffman Coding                   |
|---------------------|---------------------------------|----------------------------------|
| Problem type        | Scheduling/optimization         | Data compression                 |
| Input               | Jobs, deadlines, profits        | Symbols and frequencies          |
| Goal                | Maximize total profit           | Minimize encoded data length     |
| Greedy choice       | Select highest-profit job       | Merge two least-frequent symbols |
| Main data structure | Array / sorting                 | Min-heap + binary tree           |
| Output              | Job schedule and maximum profit | Huffman codes                    |
| Greedy approach     | Local profit maximization       | Local frequency minimization     |

### 2. Job Sequencing with Deadlines

#### Greedy Strategy

1. Sort jobs in decreasing order of profit.
2. Take the highest-profit job.
3. Place it in the latest available slot before its deadline.
4. Continue until all jobs are considered.

#### Steps

Sorted order:

**J1(100), J3(27), J4(25), J2(19), J5(15)**

Available slots: **1, 2, 3**

- J1 → deadline 2 → place in slot 2
- J3 → deadline 2 → place in slot 1

- J4 → deadline 1 → no free slot
- J2 → deadline 1 → no free slot
- J5 → deadline 3 → place in slot 3

### Final Schedule

| Slot 1 | Slot 2 | Slot 3 |
|--------|--------|--------|
| J3     | J1     | J5     |

Maximum Profit =  $27 + 100 + 15 = 142$

### Correctness

The algorithm chooses the most profitable available job first and places it as late as possible. Placing a job in the latest available slot preserves earlier slots for jobs having smaller deadlines.

Therefore, the greedy strategy produces an optimal maximum-profit schedule.

## 3. Huffman Coding

Huffman Coding is used for lossless data compression.

### Greedy Strategy

1. Insert all frequencies into a min-heap.
2. Select the two symbols with the smallest frequencies.
3. Merge them into a new node.
4. Insert the new node back into the heap.
5. Repeat until only one node remains.
6. Assign 0 and 1 to the tree branches.

### Steps

$$5 + 9 = 14$$

$$12 + 13 = 25$$

$$14 + 16 = 30$$

$$25 + 30 = 55$$

$$45 + 55 = 100$$

The resulting tree gives shorter codes to frequent symbols and longer codes to less frequent symbols.

One possible coding is:

| Symbol | Frequency | Huffman Code |
|--------|-----------|--------------|
| F      | 45        | 0            |
| C      | 12        | 100          |
| D      | 13        | 101          |
| E      | 16        | 111          |
| A      | 5         | 1100         |
| B      | 9         | 1101         |

The exact 0/1 codes can differ depending on how ties are handled, but the code lengths and optimal compression cost remain equivalent.

### Correctness

Huffman Coding repeatedly combines the two least-frequent symbols. These symbols can safely be placed at the deepest level of an optimal prefix-code tree.

By repeatedly applying this greedy choice, Huffman constructs an optimal prefix-free binary code with minimum weighted path length.

## 4. Direct Greedy Comparison

| Aspect             | Job Sequencing         | Huffman Coding                       |
|--------------------|------------------------|--------------------------------------|
| Greedy choice      | Highest-profit job     | Two lowest-frequency nodes           |
| Choice direction   | Maximum first          | Minimum first                        |
| Objective          | Maximize profit        | Minimize encoded length              |
| Data structure     | Sorted list + slots    | Min-heap                             |
| Decision           | Where to schedule job  | Which nodes to merge                 |
| Result             | Optimal schedule       | Optimal prefix code                  |
| Main concept       | Exchange/slot argument | Greedy-choice + optimal-substructure |
| Repeated operation | Place job              | Merge nodes                          |
| Typical complexity | $O(n \log n)$          | $O(n \log n)$                        |
| Space              | $O(n)$                 | $O(n)$                               |

## 5. Correctness Comparison

### Job Sequencing

Its correctness is based on the idea that selecting a higher-profit job and placing it as late as possible does not reduce the possibility of scheduling other jobs.

Thus, the locally best profitable choice leads to the globally maximum profit.

### Huffman Coding

Its correctness is based on the fact that the two least-frequent symbols can be siblings at the deepest level of an optimal Huffman tree.

Merging them into one combined node reduces the problem to a smaller equivalent problem.

Thus, repeated greedy choices produce an optimal prefix code.

## 6. Time and Space Complexity

### Job Sequencing

If  $n$  jobs are given:

- Sorting jobs:  $O(n \log n)$
- Finding slots using a simple array:  $O(n^2)$  in the worst case
- Space:  $O(n)$

With an efficient Disjoint Set Union implementation for slot management:

- Time: approximately  $O(n \log n)$
- Space:  $O(n)$

### Huffman Coding

For  $n$  symbols:

- Building min-heap:  $O(n)$
- Repeated extraction and insertion:  $O(n \log n)$
- Space:  $O(n)$

Therefore:

Huffman Coding =  $O(n \log n)$

## 7. Real-World Applications

### Job Sequencing

Used in:

- CPU task scheduling
- Manufacturing production planning
- Project management
- Delivery scheduling
- Advertisement scheduling
- Cloud computing task allocation

## **Huffman Coding**

Used in:

- File compression
- Text compression
- ZIP-type compression systems
- Image compression systems
- Multimedia encoding
- Communication systems

## **8. Key Insights About the Greedy Paradigm**

1. Greedy algorithms make a locally optimal decision at each step.
2. The local choice must have a mathematical justification for producing a global optimum.
3. Job Sequencing uses profit-based selection.
4. Huffman Coding uses frequency-based merging.
5. Both problems have optimal substructure.
6. Greedy algorithms can be much faster and simpler than exhaustive search.
7. A greedy approach should not be used blindly; the greedy-choice property must be proved.
8. Different problems require different greedy rules:
  - Job Sequencing → highest profit
  - Huffman → two smallest frequencies

**Comparison Table: Job Sequencing vs Huffman Coding**

| Parameter               | Job Sequencing with Deadlines                                                               | Huffman Coding                                                                |
|-------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Problem Type            | Scheduling and optimization                                                                 | Data compression                                                              |
| Main Objective          | Maximize total profit                                                                       | Minimize total encoded length                                                 |
| Input                   | Jobs, deadlines, and profits                                                                | Symbols and their frequencies                                                 |
| Greedy Strategy         | Select the job with the highest profit first                                                | Select the two symbols with the lowest frequencies                            |
| Greedy Choice           | Highest-profit job                                                                          | Two least-frequent nodes                                                      |
| Data Structure          | Array / sorting                                                                             | Min-heap / priority queue                                                     |
| Main Operation          | Place job in the latest free slot before its deadline                                       | Merge two minimum-frequency nodes                                             |
| Output                  | Optimal job schedule and maximum profit                                                     | Optimal prefix codes                                                          |
| Correctness             | Choosing high-profit jobs and scheduling them as late as possible preserves available slots | Combining the two least-frequent symbols produces an optimal prefix-code tree |
| Optimality              | Maximizes total profit                                                                      | Minimizes weighted code length                                                |
| Time Complexity         | $O(n \log n)$ with efficient slot management                                                | $O(n \log n)$                                                                 |
| Space Complexity        | $O(n)$                                                                                      | $O(n)$                                                                        |
| Greedy Property         | Greedy-choice property and optimal substructure                                             | Greedy-choice property and optimal substructure                               |
| Type of Optimization    | Maximization                                                                                | Minimization                                                                  |
| Real-World Applications | Task scheduling, production planning, CPU scheduling, project management                    | File compression, text compression, communication, multimedia encoding        |
| Example                 | Select jobs J1, J3, J5 to obtain maximum profit                                             | Assign shorter codes to frequently occurring symbols                          |



| Parameter          | Job Sequencing with Deadlines            | Huffman Coding                      |
|--------------------|------------------------------------------|-------------------------------------|
| Final Result       | Maximum possible profit                  | Minimum possible encoded size       |
| Nature of Decision | Which job should be scheduled and where? | Which two nodes should be combined? |